

Building a general purpose RISC CPU from discrete logic components

1 Introduction

Within this project, the student develops and builds a simple RISC CPU. The device shall be used as a visual aid for future lectures ("Betriebssysteme", "Rechnerarchitektur" and others). The device shall be built from discrete logic components. The device shall be able to execute demonstrative algorithms such as primality testing, the Fibonacci series or CRC checksums. The device shall be as simple as possible and as complex as necessary.

2 Approach

2.1 Instruction Set Architecture

The student must research existing solutions within the problem space. This includes existing CPUs, ISAs and projects implementing CPUs using discrete components. It also involves critically examining the requirements of the project and comparing existing solutions for suitability. Following this initial research, the student must develop a RISC ISA. The student may either pick an existing ISA or develop a custom one. All design decisions must be recorded in the documentation.

2.2 Logic design

The CPU must be designed in detail, down to the level of individual logic components. All logic must be implemented using discrete components. Off-the-shelf ICs can be used for trivially repetitive circuits such as RAM/ROM or (de-)multiplexers. The student must also consider and allow the possibility of future expansion:

- Increasing register count
- Increasing memory size
- External IO (GPIO)

2.3 Implementation

Before physically building the device, the student must prove the designs viability by implementing a full-scale simulation. This may be done either by using logic simulators such as Logisim, BOOLR, Digital or similar software, or by implementing the design within an FPGA using a HDL. Once verified, the student must design and implement an electrical circuit implementing the ISA. The implementation must be powered by a commonly available connector.

3 Constraints & Requirements

3.1 ISA

- No interrupts
- Load/Store architecture
- Supported operations (minimum):
 - Memory access (Load, Store, Move)
 - Flow control (Jump, Conditionals)
 - (Un-)signed addition (optional Carry)
 - (Un-)signed subtraction (optional Borrow)
 - Bitwise AND
 - Bitwise OR
 - Bitwise NOT
 - Shift Left
 - Shift Right

3.2 Physical

- Dimensions (maximum): $500\text{ mm} \times 500\text{ mm} \times 100\text{ mm}$
- Clock speed (minimum): $1\text{ Hz} - 10\text{ kHz}$

3.3 Functional

- **Control user interface:**

The user must be able to control the state of the CPU externally. This may be realized either as a set of buttons or an external connector for other electrical systems. The interface must allow the user to:

 - reset the CPU state
 - Enable/disable the CPU
 - Control the clock speed (Manual, $1\text{ Hz} - 10\text{ kHz}$)
- **Visual interface:**

The user must be able to visually identify the state of the CPU. This may be achieved

by optical components such as LEDs or 7-Segment-displays. The user must be able to see the following:

- contents of latches/registers
- contents of buses
- execution mode (run/halt)
- conditional flags
- current instruction opcode

- **Programming the device:**

The user must be able to change the programming of the CPU. If an existing ISA is implemented, an existing compiler may be used. If a custom ISA is designed, software must be supplied to generate executable programs from assembly code. The compiler must support all CPU features. The user must be able to install the program in program ROM via an external interface.